PrimeRx Case Study: Code & Analytical Approach (MySQL)

This document outlines the step-by-step approach and the exact **MySQL** queries used to solve each of the 7 business questions in the PrimeRx Commercial Opportunity Analysis case study.

---

0. Data Setup and Assumptions

Approach: We assume the data from the three Excel sheets has been loaded into three relational database tables: `1. Alerts (Columns: Alert_ID, Alert_Date, HCP_ID, Lab_Result) 2. Sales (Columns: Prescription_ID, Prescription_Date, HCP_ID, Drug_Name, Drug_ID, Prescription_Volume) 3. Affiliation (Columns: HCP_ID, Account_ID, Account_Name)`

Data types for dates should be standard `DATE` or `DATETIME`. `HCP_ID` fields should ideally be cleaned (trimmed and uppercase) during the ETL load process before running these queries.

Question 1: Who are the most active doctors?

Approach: For **Ranking 1 (Alert Volume)**, we use a simple `GROUP BY` on the `Alerts` table. For **Ranking 2 (Prescription Volume)**, we do an `INNER JOIN` against a subquery of distinct `HCP_ID`s from the `Alerts` table to ensure we only count prescription volume from our "alerts universe".

```
-- Ranking 1: Top 10 by Alert Volume
SELECT HCP_ID, COUNT(*) AS Total_Alerts
FROM Alerts
GROUP BY HCP_ID
ORDER BY Total_Alerts DESC
LIMIT 10;


-- Ranking 2: Top 10 by Prescription Volume (filtered to alerts universe)
SELECT s.HCP_ID, SUM(s.Prescription_Volume) AS Total_Rx_Volume
FROM Sales s
INNER JOIN (
    SELECT DISTINCT HCP_ID
    FROM Alerts
) a ON s.HCP_ID = a.HCP_ID
GROUP BY s.HCP_ID
ORDER BY Total_Rx_Volume DESC
LIMIT 10;
```

Question 2: Which doctors see the need but don't act on it?

Approach: We use a Common Table Expression (CTE) to find the first positive alert date and total positive alerts for each doctor. We then `LEFT JOIN` to the `Sales` table, specifically looking for prescriptions within a 30-day window

(DATE_ADD function) after the first alert. We use `HAVING SUM(...) = 0` to filter for doctors who did *not* write a prescription in that window.

```
WITH FirstPosAlerts AS (
    SELECT
        HCP_ID,
        MIN(Alert_Date) AS First_Positive_Date,
        COUNT(*) AS Positive_Alert_Count
    FROM Alerts
    WHERE Lab_Result = 'Positive'
    GROUP BY HCP_ID
)
SELECT
    f.HCP_ID,
    f.Positive_Alert_Count,
    f.First_Positive_Date
FROM FirstPosAlerts f
LEFT JOIN Sales s
    ON f.HCP_ID = s.HCP_ID
    AND s.Prescription_Date BETWEEN f.First_Positive_Date AND DATE_ADD(f.First_Positive_Date, INTERVAL 30
DAY)
GROUP BY f.HCP_ID, f.Positive_Alert_Count, f.First_Positive_Date
HAVING SUM(COALESCE(s.Prescription_Volume, 0)) = 0
ORDER BY f.Positive_Alert_Count DESC
LIMIT 10;
```

Question 3: Prescription mix for Top 10 alert-volume HCPs

Approach: We identify the top 10 HCPs using a CTE, then join that to the `Sales` table. We use conditional aggregation (SUM(CASE WHEN...)) to pivot the volumes into "Our Drug" (`DRG001`) and "Competitor" buckets. Finally, we calculate the percentage split.

```
WITH Top10Alerts AS (
    SELECT HCP_ID
    FROM Alerts
    GROUP BY HCP_ID
    ORDER BY COUNT(*) DESC
    LIMIT 10
),
SalesSummary AS (
    SELECT
        s.HCP_ID,
        SUM(CASE WHEN s.Drug_ID = 'DRG001' THEN s.Prescription_Volume ELSE 0 END) AS Our_Drug_Volume,
        SUM(CASE WHEN s.Drug_ID != 'DRG001' THEN s.Prescription_Volume ELSE 0 END) AS Competitor_Volume
    FROM Sales s
    INNER JOIN Top10Alerts t ON s.HCP_ID = t.HCP_ID
    GROUP BY s.HCP_ID
)
SELECT
    HCP_ID,
    Competitor_Volume,
    Our_Drug_Volume,
    (Our_Drug_Volume + Competitor_Volume) AS Total_Volume,
    (Our_Drug_Volume / (Our_Drug_Volume + Competitor_Volume) * 100) AS `Our_Drug_%`,
    (Competitor_Volume / (Our_Drug_Volume + Competitor_Volume) * 100) AS `Competitor_%`
FROM SalesSummary;
```

Question 4: Top 10 accounts by conversion ratio

Approach: We create one CTE for rolling up total alerts by `Account_ID` byjoining Alerts to Affiliation. We create a second CTE for rolling up DRG001 prescription volumeby `Account_ID`. We then join them together and divide Rx Volume by Alert Count to calculate the Conversion Ratio.

```
WITH AccountAlerts AS (
    SELECT aff.Account_ID, aff.Account_Name, COUNT(a.Alert_ID) AS Total_Alerts
    FROM Alerts a
    INNER JOIN Affiliation aff ON a.HCP_ID = aff.HCP_ID
    GROUP BY aff.Account_ID, aff.Account_Name
),
AccountSales AS (
    SELECT aff.Account_ID, SUM(s.Prescription_Volume) AS DRG001_Rx
    FROM Sales s
    INNER JOIN Affiliation aff ON s.HCP_ID = aff.HCP_ID
    WHERE s.Drug_ID = 'DRG001'
    GROUP BY aff.Account_ID
)
SELECT
    aa.Account_Name,
    aa.Total_Alerts,
    COALESCE(asales.DRG001_Rx,        0)        AS        DRG001_Rx_Count,
    (COALESCE(asales.DRG001_Rx, 0) / aa.Total_Alerts) AS Conversion_Ratio
FROM AccountAlerts aa
LEFT JOIN AccountSales asales ON aa.Account_ID = asales.Account_ID
ORDER BY Conversion_Ratio ASC
LIMIT 10;
```

Question 5: Account size vs Rx-per-active-doctor ratio

Approach: We calculate the total number of doctors per account. We then count the DISTINCT doctors who have written a prescription for DRG001 (Active_Prescribers), and sum the volume. We divide the total volume by the active prescribers count, filtering out accounts where Active_Prescribers = 0.

```
WITH AccountDocs AS (
    SELECT Account_ID, COUNT(DISTINCT HCP_ID) AS Total_Doctors
    FROM Affiliation
    GROUP BY Account_ID
),
ActiveDocs AS (
    SELECT
        aff.Account_ID,
        COUNT(DISTINCT s.HCP_ID) AS Active_Prescribers,
        SUM(s.Prescription_Volume) AS Total_DRG001_Rx
    FROM Sales s
    INNER JOIN Affiliation aff ON s.HCP_ID = aff.HCP_ID
    WHERE s.Drug_ID = 'DRG001'
    GROUP BY aff.Account_ID
)
SELECT
    ad.Account_ID,
    ad.Total_Doctors,
    COALESCE(act.Active_Prescribers, 0) AS Active_Prescribers,
    COALESCE(act.Total_DRG001_Rx, 0) AS Total_DRG001_Rx,
    (COALESCE(act.Total_DRG001_Rx, 0) / act.Active_Prescribers) AS Rx_Per_Active_Doctor
FROM AccountDocs ad
LEFT JOIN ActiveDocs act ON ad.Account_ID = act.Account_ID
WHERE act.Active_Prescribers > 0
ORDER BY Rx_Per_Active_Doctor ASC
LIMIT 10;
```

Question 6: Behavioral Lift & Segmentation

Approach: We use a CTE to grab the first positive alert. A second CTE aggregates sales into 90-day pre and post windows using DATE_SUB and DATE_ADD. A final CTE calculates the lift percentage and assigns the behavioral segment (New Starter, Grower, Non-responder) using CASE WHEN logic.

```
WITH FirstPos AS (
    SELECT HCP_ID, MIN(Alert_Date) AS First_Pos_Date
    FROM Alerts
    WHERE Lab_Result = 'Positive'
    GROUP BY HCP_ID
),
SalesWindows AS (
    SELECT
        f.HCP_ID,
        f.First_Pos_Date,
        SUM(CASE WHEN s.Prescription_Date >= DATE_SUB(f.First_Pos_Date, INTERVAL 90 DAY)
                 AND s.Prescription_Date < f.First_Pos_Date THEN s.Prescription_Volume ELSE 0 END) AS
Pre_90d_Rx,
        SUM(CASE WHEN s.Prescription_Date > f.First_Pos_Date
                 AND s.Prescription_Date <= DATE_ADD(f.First_Pos_Date, INTERVAL 90 DAY) THEN
s.Prescription_Volume ELSE 0 END) AS Post_90d_Rx
    FROM FirstPos f
    LEFT JOIN Sales s ON f.HCP_ID = s.HCP_ID AND s.Drug_ID = 'DRG001'
    GROUP BY f.HCP_ID, f.First_Pos_Date
),
LiftCalc AS (
    SELECT
        HCP_ID,
        Pre_90d_Rx,
        Post_90d_Rx,
        CASE
            WHEN Pre_90d_Rx = 0 AND Post_90d_Rx > 0 THEN NULL
            WHEN Pre_90d_Rx = 0 AND Post_90d_Rx = 0 THEN 0.0
            ELSE ((Post_90d_Rx - Pre_90d_Rx) / Pre_90d_Rx) * 100
        END AS Lift_Pct,
        CASE
            WHEN Pre_90d_Rx = 0 AND Post_90d_Rx > 0 THEN 'New Starter'
            WHEN Pre_90d_Rx = 0 AND Post_90d_Rx = 0 THEN 'Non-responder'
            WHEN (((Post_90d_Rx - Pre_90d_Rx) / Pre_90d_Rx) * 100) >= 20.0 THEN 'Grower'
            ELSE 'Non-responder'
        END AS Segment
    FROM SalesWindows
)
SELECT * FROM LiftCalc
ORDER BY Lift_Pct DESC;
```

Question 7: Field Force Allocation (Capacity Math)

Approach: This step does not strictly require a database query, as the math is qualitative resource allocation based on the segments identified in Question 6. However, we can calculate the total available capacity via standard arithmetic and map it to the segment distribution from the query above.

```
Total Capacity Math:
40 reps * 8 calls/day * 20 days/month * 3 months = 19,200 quarterly calls

Strategy mapping based on ROI per call:
1. New Starters (~50% allocation): 9,600 calls
2. Growers (~35% allocation): 6,720 calls
3. Non-responders (~15% allocation): 2,880 calls
```